

MEMORIA PROYECTO INTERMODULAR

Forecasting de Demanda en Retail

Sistema end-to-end de predicción de ventas con XGBoost, Prophet y SARIMAX · Pipeline ETL reproducible con Docker, PostgreSQL y MLflow

Víctor García Garrido
Francisco Javier Fernández

Tutor: Willman Acosta

Máster en Inteligencia Artificial y Big Data · Campus Cámara de Comercio EUSA · Sevilla, Mayo 2026



1. Introducción:

1.1 Idea general y contexto

La gestión eficiente del inventario constituye uno de los principales retos operativos del sector retail. Anticipar con precisión cuánto se va a vender en los próximos días permite a las empresas reducir roturas de stock, minimizar el exceso de mercancía y optimizar sus cadenas de suministro. Sin embargo, la demanda en retail está condicionada por múltiples factores simultáneos: estacionalidad semanal y anual, campañas promocionales, días festivos y tendencias de largo plazo, lo que hace que los métodos tradicionales basados en medias históricas resulten insuficientes.

En este contexto, el presente Trabajo de Fin de Máster propone el diseño y desarrollo de un sistema end-to-end de predicción de demanda aplicado a un conjunto de tiendas de retail. El sistema es capaz de ingestar datos históricos de ventas, construir variables predictoras relevantes, entrenar y comparar múltiples modelos de forecasting y, finalmente, generar predicciones automáticas semanales para un horizonte de cuatro semanas. Todo ello de forma reproducible, trazable y desplegable en un entorno dockerizado.

El sistema se desarrolla sobre el dataset público *Store Sales Dataset* [1], disponible en Kaggle bajo licencia Apache 2.0. Este dataset contiene registros diarios de ventas simuladas para 10 tiendas minoristas a lo largo de dos años completos (enero 2022 – diciembre 2023), con un total de 7.300 registros. Cada registro incluye el identificador de tienda, el volumen de ventas diario en rupias (₹), una variable binaria que indica si ese día estaba activa una promoción y otra que señala si correspondía a un día festivo. Esta estructura hace del dataset un entorno idóneo para entrenar, validar y comparar modelos de series temporales con variables exógenas.

10	7.300	2 años
TIENDAS	REGISTROS	2022 – 2023

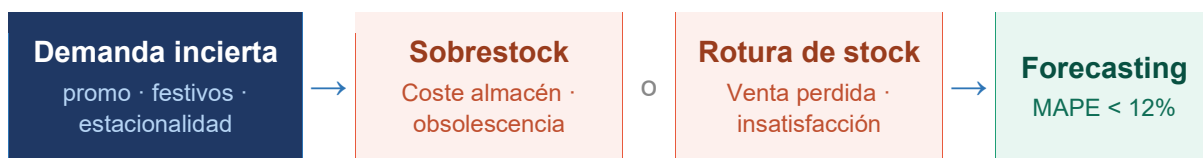


1.2 Definición del problema

Las cadenas de retail se enfrentan de forma recurrente a un dilema de planificación: pedir demasiado stock implica costes de almacenamiento y riesgo de obsolescencia, mientras que pedir poco genera roturas que se traducen directamente en ventas perdidas y clientes insatisfechos. Este problema se agrava cuando la demanda es volátil, cuando existen múltiples tiendas con comportamientos heterogéneos y cuando las promociones distorsionan los patrones históricos habituales.

El problema que aborda este proyecto se puede enunciar de forma concisa: *¿es posible anticipar, con un error inferior al 12% (MAPE), las ventas diarias de cada tienda para las próximas cuatro semanas, considerando el efecto de promociones y festivos?*

El alcance del proyecto queda delimitado de la siguiente manera: el sistema trabaja con granularidad diaria a nivel de tienda, el horizonte de predicción es de 28 días y la evaluación se realiza tanto de forma global como segmentada por tienda y por régimen de demanda (con promoción y sin promoción). Se excluye del alcance la integración con sistemas ERP reales y el consumo de datos en tiempo real.



1.3 Objetivo general

Diseñar y desarrollar un sistema end-to-end de forecasting de demanda para tiendas de retail que, a partir de datos históricos de ventas diarias con variables exógenas (promociones y festivos), sea capaz de generar predicciones semanales automatizadas con un error porcentual medio absoluto (MAPE) inferior al 12%, de forma reproducible, trazable y desplegable mediante contenedores Docker.



Pipeline end-to-end · Sistema de forecasting de demanda



1.4 Objetivos específicos

Para alcanzar el objetivo general, el proyecto se articula en los siguientes objetivos específicos, ordenados de forma secuencial y coherente con las fases de desarrollo:

1. **Construir un pipeline de ingesta y curación de datos** que cargue el dataset de ventas en una base de datos PostgreSQL, resuelva nulos y duplicados, valide la continuidad temporal y garantice que más del 90% de los registros originales superan los controles de calidad definidos mediante Great Expectations.
2. **Realizar un análisis exploratorio de los datos (EDA)** que permita identificar los patrones de estacionalidad semanal y anual, detectar anomalías y outliers, cuantificar el impacto de las promociones y los festivos sobre la demanda, y seleccionar las tiendas piloto más representativas para el modelado.
3. **Desarrollar un módulo de feature engineering** que construya variables predictoras relevantes: retardos temporales (lags), medias móviles, indicadores de calendario (día de la semana, mes, festivo, estación) y variables exógenas (indicador de promoción), almacenando el resultado en una tabla de características lista para el entrenamiento.
4. **Entrenar y comparar cuatro modelos de forecasting** (Seasonal Naïve como baseline, XGBoost, Prophet y SARIMAX) utilizando una estrategia de validación expanding window para evitar el sesgo de lookahead, registrando todos los experimentos y métricas en MLflow.
5. **Seleccionar el modelo de mejor rendimiento** mediante un análisis comparativo del MAPE global y segmentado (por tienda y por régimen de demanda), incluyendo análisis de residuos, y documentar la decisión de forma razonada y reproducible.
6. **Implementar un sistema de serving automatizado** que ejecute el modelo seleccionado de forma semanal, almacene las predicciones a 28 días con intervalos de confianza en base de datos y proponga un mecanismo de monitorización basado en data drift y MAPE rolling que permita detectar degradación del modelo en producción.



Nº	Objetivo específico	Métrica de éxito
1	Construir un pipeline de ingesta y curación de datos en PostgreSQL con validación automática.	>90% filas originales conservadas · 0 nulos · 0 duplicados
2	Realizar un análisis exploratorio (EDA) que identifique patrones de estacionalidad, anomalías e impacto de promociones.	Notebook EDA entregado con SKU piloto seleccionado y justificado
3	Desarrollar un módulo de feature engineering con lags, medias móviles, calendario y variables exógenas.	Tabla analytics.features sin nulos, disponible para entrenamiento
4	Entrenar y comparar cuatro modelos (Seasonal Naïve, XGBoost, Prophet, SARIMAX) con validación expanding window.	Todos los experimentos registrados en MLflow con MAPE calculado
5	Seleccionar el modelo de mejor rendimiento con justificación técnica y análisis de residuos.	MAPE global < 12% en el conjunto de test
6	Implementar un batch predictor semanal con monitorización de data drift y MAPE rolling.	Predicciones a 28 días almacenadas en analytics.predictions

2. Marco teórico:

En esta sección se construye el andamiaje conceptual y técnico del proyecto. Se presentan los conceptos fundamentales de las series temporales y el forecasting, los modelos de predicción empleados, las tecnologías y herramientas seleccionadas, y el estado del arte en sistemas de predicción de demanda en retail. Cada apartado está alineado con los objetivos específicos definidos en la sección anterior.

2.1 Forecasting de demanda - conceptos clave

El forecasting de demanda es una disciplina que combina métodos estadísticos y de aprendizaje automático para anticipar valores futuros de una variable de interés a partir de su historia pasada y de factores externos relacionados. En el contexto del retail, la variable objetivo es el volumen de ventas diario por tienda [2].



Una serie temporal es una secuencia de observaciones indexadas en el tiempo. Su análisis requiere identificar cuatro componentes principales: tendencia (dirección a largo plazo), estacionalidad (patrones cíclicos de período fijo), ciclo (fluctuaciones de período variable) y ruido (variabilidad aleatoria no explicada). La descomposición de estos componentes, mediante técnicas como STL [3], es el punto de partida del análisis exploratorio de datos (EDA) de este proyecto.

La métrica de evaluación principal utilizada en este trabajo es el MAPE (Mean Absolute Percentage Error), definido como la media del error porcentual absoluto entre el valor real y el predicho. Su ventaja respecto a métricas absolutas como el RMSE reside en su interpretabilidad: un MAPE del 10% significa que el modelo se equivoca, en promedio, un 10% respecto al valor real, con independencia de la escala de ventas de cada tienda [4]. El objetivo del proyecto es alcanzar un MAPE global inferior al 12% en el conjunto de test.

Concepto	Definición
Serie temporal	Secuencia de observaciones ordenadas cronológicamente. En este proyecto, las ventas diarias por tienda.
Estacionalidad	Patrón que se repite con una frecuencia fija (semanal, anual). Detectada en el EDA mediante descomposición STL.
MAPE	Mean Absolute Percentage Error. Métrica principal de evaluación: error porcentual medio entre valor real y predicho.
Lag (retardo)	Variable que representa el valor de la serie en un instante anterior (e.g. ventas de hace 7 días).
Media móvil (MA)	Promedio de los últimos N valores. Suaviza la señal y reduce el ruido en la serie.
Expanding window	Estrategia de validación donde el conjunto de entrenamiento crece en cada iteración, evitando lookahead bias.
Feature engineering	Proceso de construcción de variables predictoras a partir de los datos brutos para alimentar los modelos.
Data drift	Cambio en la distribución estadística de los datos de entrada con respecto al histórico de entrenamiento.
SHAP values	Método de explicabilidad basado en la teoría de juegos que cuantifica la contribución de cada variable a la predicción.

2.2 Modelos utilizados



El sistema implementa y compara cuatro modelos de predicción, seleccionados para cubrir un espectro amplio de enfoques: desde métodos estadísticos clásicos hasta técnicas de aprendizaje automático. Esta diversidad permite identificar qué familia de modelos se adapta mejor a las características del dataset y justificar la elección del modelo final con evidencia empírica.

2.2.1 Seasonal Naïve (baseline)

El modelo Seasonal Naïve constituye el punto de referencia mínimo del sistema. Su predicción para el instante t es simplemente el valor observado en el mismo período de la semana anterior: $\hat{y}(t) = y(t - 7)$. A pesar de su simplicidad, este modelo es difícil de superar en series con estacionalidad fuerte y baja tendencia [4]. En este proyecto se utiliza como baseline obligatorio: cualquier modelo más complejo debe justificar su coste computacional superando su MAPE (objetivo: 15–20% para el baseline, <12% para el modelo final).

2.2.2 SARIMAX

El modelo SARIMAX (Seasonal AutoRegressive Integrated Moving Average with eXogenous variables) extiende el clásico ARIMA incorporando componentes estacionales y variables exógenas [5]. Se parametriza como SARIMAX(p, d, q)(P, D, Q, s), donde p, d, q son los órdenes autorregresivo, de diferenciación e integración no estacional, y P, D, Q, s sus equivalentes estacionales con período s . La selección de órdenes óptimos se realiza minimizando el criterio AIC/BIC. En este proyecto, las variables exógenas incorporadas son el indicador de promoción (promo) y el indicador de festivo (holiday), implementado mediante la librería statsmodels [6].

2.2.3 XGBoost

XGBoost (eXtreme Gradient Boosting) es un algoritmo de aprendizaje automático basado en la construcción iterativa de árboles de decisión [7]. En cada iteración, un nuevo árbol aprende a corregir los errores del conjunto anterior, minimizando una función de pérdida mediante descenso de gradiente. Sus principales ventajas para el forecasting son su capacidad de capturar relaciones no lineales entre variables, su eficiencia computacional y la disponibilidad de herramientas de explicabilidad como los valores SHAP [8].

Dado que XGBoost no es un modelo nativo de series temporales, requiere una etapa previa de feature engineering que transforme la serie en un problema de regresión supervisada. Para ello se construyen retardos temporales (lags), medias móviles y



variables de calendario que codifican implícitamente la estructura temporal de los datos. La optimización de hiperparámetros (`learning_rate`, `max_depth`, `n_estimators`) se realiza mediante Hyperopt.

2.2.4 Prophet

Prophet es un modelo de forecasting desarrollado por Meta (anteriormente Facebook) [9], basado en la descomposición aditiva de la serie en tres componentes: tendencia (modelada con una función logística o lineal con `change points`), estacionalidad (representada mediante series de Fourier) y efectos de festivos (modelados como variables binarias con ventana temporal). Su principal fortaleza es la gestión automática de cambios de tendencia y la incorporación nativa de calendarios de festivos, lo que lo hace especialmente adecuado para series de retail con efectos promocionales marcados. En este proyecto se añade el indicador de promoción como regresor externo adicional.

Tabla 4. Comparativa de los cuatro modelos implementados en el sistema.

Modelo	Tipo	Ventajas	Limitaciones	Uso en este TFM
SNaïve	Estadístico – baseline	Sin entrenamiento, reproducible al 100%	No captura tendencia ni exógenas	Referencia mínima (MAPE 15-20%)
SARIMAX	Estadístico – series temporales	Explica componentes (tendencia, ciclo)	Requiere estacionariedad; lento con muchos SKUs	Modelo clásico de referencia avanzada
XGBoost	ML – gradient boosting	Captura no-linealidades; feature importance	No nativo para series temporales; requiere features	Modelo principal ML con SHAP
Prophet	Bayesiano – aditivo	Gestiona festivos y <code>change points</code> automáticamente	Menos preciso en series con alta volatilidad	Modelo alternativo para estacionalidad compleja

2.3 Tecnología y herramientas



El sistema se construye íntegramente sobre tecnologías de código abierto, lo que garantiza reproducibilidad, transparencia y coste cero en licencias. La selección de cada herramienta responde a criterios técnicos concretos y a su adopción extendida en proyectos de MLOps en entornos de producción.

La base de datos elegida es PostgreSQL [10], un sistema relacional maduro y ampliamente adoptado en entornos de datos empresariales. Se organiza en dos esquemas diferenciados: staging (datos en bruto tal como se ingestan) y analytics (datos curados, features y predicciones). Esta separación sigue el patrón de arquitectura medallion [11], que facilita la trazabilidad y la auditoría del pipeline.

El entorno de ejecución se contenedoriza con Docker y Docker Compose [12], garantizando que el sistema sea completamente reproducible en cualquier máquina con independencia del sistema operativo o las dependencias instaladas. El orquestador de flujos es Prefect [13], que permite programar la ejecución semanal del batch predictor y gestionar reintentos automáticos en caso de fallo.

El tracking de experimentos se centraliza en MLflow [14], que registra para cada ejecución los hiperparámetros, las métricas de evaluación (MAPE global y por segmento) y los artefactos del modelo entrenado. Esto permite comparar objetivamente los cuatro modelos y reproducir cualquier experimento pasado. La calidad de los datos en la capa de curación se valida automáticamente mediante Great Expectations [15], que define y ejecuta un conjunto de checks (sin nulos, sin duplicados, continuidad temporal) como parte del pipeline ETL.

Tabla 5. Stack tecnológico del sistema y justificación de cada herramienta.

Herramienta	Categoría	Versión / Licencia	Rol en el proyecto
PostgreSQL	Base de datos relacional	v15 · Open Source	Almacén central: staging, analytics, predictions
Docker / Compose	Contenedorización	v24 · Apache 2.0	Reproducibilidad del entorno completo
MLflow	MLOps – tracking	v2.x · Apache 2.0	Registro de experimentos, parámetros y métricas
Prefect	Orquestación de flujos	v2.x · Apache 2.0	Automatización del pipeline semanal de predicción



Herramienta	Categoría	Versión / Licencia	Rol en el proyecto
Great Expectations	Calidad de datos	v0.18 · Apache 2.0	Validación automática del dataset curado
Python 3.11	Lenguaje principal	v3.11 · PSF License	Toda la lógica ETL, modelado y serving
statsmodels	Librería estadística	v0.14 · BSD	Implementación de SARIMAX
XGBoost	Librería ML	v2.x · Apache 2.0	Modelo de gradient boosting
Prophet	Librería forecasting	v1.1 · MIT	Modelo bayesiano con estacionalidad
SHAP	Explicabilidad ML	v0.44 · MIT	Análisis de importancia de variables

2.4 Estado del arte

La predicción de demanda en retail es uno de los campos de aplicación del machine learning con mayor recorrido industrial. En la última década, la disponibilidad de datos históricos de ventas a granularidad diaria o incluso horaria ha impulsado el desarrollo de sistemas cada vez más sofisticados, que van desde los modelos estadísticos clásicos hasta las arquitecturas de deep learning especializadas en series temporales.

Entre las soluciones comerciales más extendidas se encuentran Amazon Forecast, que ofrece modelos preentrenados accesibles mediante API REST, y Azure Machine Learning Forecasting, integrado en el ecosistema de Microsoft. Ambas plataformas permiten obtener predicciones con MAPE típicos del 8–15% sin necesidad de implementar ni mantener modelos propios [16]. Sin embargo, presentan inconvenientes significativos para entornos que requieren control total del proceso: dependencia del proveedor (vendor lock-in), coste variable por volumen de predicciones y opacidad en el proceso de modelado.

En el ámbito open source, la librería Nixtla StatsForecast [17] ofrece implementaciones altamente optimizadas de modelos estadísticos clásicos (ARIMA, ETS, Theta) con tiempos de entrenamiento muy reducidos. Por su parte, GluonTS [18] y DeepAR proporcionan arquitecturas de deep learning para series temporales, con resultados competitivos en datasets de gran escala, aunque requieren volúmenes de datos significativamente mayores y mayor coste computacional.



El sistema desarrollado en este TFM se diferencia de las soluciones existentes en tres aspectos clave: (1) control total sobre el pipeline, desde la ingesta hasta el serving, desplegable on-premise sin dependencia de proveedores cloud; (2) trazabilidad completa de todos los experimentos mediante MLflow; y (3) explicabilidad de las predicciones mediante valores SHAP, lo que permite identificar qué variables (promociones, festivos, lags) impulsan cada predicción y en qué medida.

Tabla 6. Comparativa del sistema propuesto frente a soluciones existentes.

Solución	Tipo	MAPE típico	Diferencia con este TFM
Amazon Forecast	SaaS / cloud propietario	8–14%	Sin control sobre el modelo; coste por predicción; sin explicabilidad
Azure ML Forecasting	SaaS / cloud propietario	9–15%	Dependencia del proveedor; difícil auditoría del pipeline
Nixtla (StatsForecast)	Open source	10–16%	Orientado a velocidad; menos énfasis en MLOps y trazabilidad
GluonTS / DeepAR	Deep learning	8–12%	Requiere grandes volúmenes de datos; caja negra; alto coste computacional
Este TFM	Open source · on-premise	Objetivo <12%	Control total, trazabilidad con MLflow, explicabilidad SHAP, coste cero en licencias

3. Marco metodológico

3.1 Metodología de desarrollo

El proyecto adopta un enfoque de desarrollo secuencial por fases con hitos Go/No-Go. La dependencia estricta entre capas del pipeline hace este enfoque más adecuado que metodologías ágiles generalistas: no es posible entrenar modelos sin datos curados ni evaluar sin features construidas previamente. El incumplimiento de un hito bloquea el avance a la fase siguiente.

El equipo se organiza por perfiles funcionales: el perfil de ingeniería de datos cubre infraestructura, ETL y serving; el perfil de ciencia de datos cubre EDA, modelado y documentación. Los puntos de contacto se limitan al schema.sql acordado al inicio del proyecto y a las reuniones semanales de coordinación. El código se gestiona con Git mediante feature branches y Pull Requests obligatorios antes de merge a main.

Tabla 14. Fases del proyecto, perfiles responsables y herramientas.



F.	Fase	Semana	Perfil	Herramientas
F1	Infra + Ingesta + Curación	Sem. 1	Ing. de Datos	<i>Docker, PostgreSQL, Great Expectations</i>
F2	EDA + Exploración	Sem. 1	Ciencia de Datos	<i>Jupyter, pandas, matplotlib</i>
F3	Feature Eng. + Baseline	Sem. 2	Ing. de Datos	<i>Python, MLflow, holidays</i>
F4	Modelado ML	Sem. 2–3	Ambos perfiles	<i>XGBoost, Prophet, SARIMAX, Hyperopt</i>
F5	Comparativa + Decisión	Sem. 3	Ambos perfiles	<i>MLflow, statsmodels, SHAP</i>
F6	Análisis segmentado + SHAP	Sem. 4	Ciencia de Datos	<i>SHAP, pandas</i>
F7	Serving + MLOps	Sem. 4	Ing. de Datos	<i>Prefect, MLflow Registry</i>
F8	Redacción y cierre	Sem. 4	Ambos perfiles	<i>pytest, black, flake8</i>

Las decisiones de arquitectura más relevantes adoptadas durante el desarrollo, junto con su justificación técnica, se recogen a continuación:

Tabla 15. Decisiones técnicas de arquitectura.

Decisión	Justificación
Granularidad diaria	Preserva señales intra-semana. La agregación semanal enmascara patrones relevantes para XGBoost.
XGBoost global + SKU como feature	730 obs/SKU insuficientes para Hyperopt individual. El modelo global opera sobre 7.230 filas (estándar M5/Walmart).
Grid AIC/BIC sobre SKU piloto	El expanding window completo tardaría >2h. El orden (0,1,2)(0,1,1,7) con AIC=597.57 se obtuvo en 14 segundos.
Sin normalización log(1+y)	El MAPE en escala logarítmica no es comparable con el MAPE en escala original. Sesgo por desigualdad de Jensen.
Modelos descartados	LSTM (datos insuficientes por SKU), ETS/Theta (sin diferenciación vs baseline), ensemble XGB+Prophet (reservado como mejora futura).

El registro completo de decisiones, hipótesis rechazadas y alternativas evaluadas se documenta en *MODELING_DECISIONS.md* del repositorio del proyecto.

3.2 Pipeline ETL

El pipeline de datos sigue una arquitectura de cinco capas que transforma el CSV original en predicciones almacenadas en base de datos. Cada capa escribe en un esquema diferenciado de PostgreSQL siguiendo el patrón medallion (staging → analytics), lo que garantiza trazabilidad y permite reproducir cualquier paso de forma independiente.

Tabla 16. Capas del pipeline ETL y resultado de cada transformación.



#	Capa	Transformación	Herramientas	Resultado
1	Ingesta	CSV → <i>staging.sales_raw</i>	pandas, SQLAlchemy	7.300 filas · logging con load_id y load_date
2	Curación	<i>staging</i> → <i>analytics.sales_curated</i>	Great Expectations, pandas	0 nulos · 0 duplicados · >90% filas originales
3	Feature Eng.	<i>curated</i> → <i>analytics.features</i>	Python, holidays	7.230 filas (-70 por lookahead bias lag_7)
4	Modelado	<i>features</i> → <i>MLflow experiments</i>	XGBoost, Prophet, statsmodels	4 experimentos registrados en MLflow
5	Predicciones	<i>modelo</i> → <i>analytics.predictions</i>	Prefect, SQLAlchemy	Horizonte 28 días · intervalos P10/P50/P90

La base de datos PostgreSQL organiza los datos en cuatro tablas con responsabilidades claramente separadas:

Tabla 17. Esquema de base de datos PostgreSQL del proyecto.

Tabla	Capa	Contenido
<i>staging.sales_raw</i>	Ingesta bruta	Datos tal como llegan del CSV. Incluye load_id, load_date y source para trazabilidad de cada carga.
<i>analytics.sales_curated</i>	Datos limpios	Dataset validado por Great Expectations: sin nulos en columnas clave, sin duplicados (store, date), continuidad temporal garantizada.
<i>analytics.features</i>	Features ML	7.230 filas con lags (1, 7, 14, 28), medias móviles (MA7, MA14, STD7), variables de calendario y promotion_flag.
<i>analytics.predictions</i>	Predicciones	Salida del batch predictor semanal. Almacena predicción P50 e intervalos de confianza P10/P90 para los próximos 28 días por tienda.

La fase de feature engineering genera 14 variables predictoras a partir del dataset curado. Las features temporales se calculan siempre con shift(1) previo al rolling para evitar data leakage. El resultado son 7.230 filas (7.300 originales menos 70 eliminadas por lookahead bias de los primeros 7 días por tienda):

Tabla 18. Features generadas y almacenadas en analytics.features.

Feature	Tipo	Descripción
<i>quantity_lag_1, lag_7 lag_14, lag_28</i>	Temporal	Retardos temporales. Capturan autocorrelación diaria, semanal, quincenal y mensual. Calculados con shift() para evitar data leakage.
<i>ma_7, ma_14, std_7</i>	Ventana móvil	Media y desviación estándar en ventanas de 7 y 14 días. Suavizan el ruido y capturan tendencia local y volatilidad.
<i>day_of_week, month quarter, is_weekend</i>	Calendario	Variables de calendario codificadas como enteros ordinales. XGBoost las gestiona nativamente sin one-hot encoding.



Feature	Tipo	Descripción
<i>is_holiday</i>	Exógena	Festivos de España obtenidos con el paquete holidays. Correlación con demanda confirmada en el EDA.
<i>promotion_flag</i>	Exógena	Variable binaria del dataset original. Factor dominante en SHAP values (media 9.15) — mayor impacto individual sobre la predicción.
<i>temperature, event_indicator</i>	Placeholder	Valores NULL en esta versión por ausencia de datos reales. Reservados para integración futura con API AEMET.

Durante el desarrollo del pipeline se registraron y resolvieron cinco incidencias técnicas documentadas a continuación como referencia para la reproducibilidad del proyecto:

Tabla 19. Errores encontrados durante el desarrollo y soluciones aplicadas.

Error	Causa	Solución
ModuleNotFoundError: holidays	Paquete holidays no instalado en el entorno.	pip3 install holidays
PermissionError: '/mlflow'	artifact_path apuntaba al path interno del contenedor.	Eliminar línea log_artifact del script.
BAD_REQUEST: invalid integer '2.'	Cliente MLflow 2.17.2 incompatible con servidor 2.12.1.	pip3 install mlflow==2.12.1 para alinear versiones.
TypeError: unexpected 'early_stopping_rounds'	Parámetro ubicado en fit() en lugar del constructor XGBRegressor.	Mover parámetro al constructor.
SARIMAX colgado >70 min	Expanding window completo con órdenes fijos sobre todos los SKUs.	Rediseñar con grid AIC/BIC solo sobre SKU piloto.

El código completo del pipeline ETL se encuentra en `src/etl/` del repositorio. Los checks de Great Expectations están definidos en `great_expectations/expectations/`.

3.3 Estrategia de validación

La evaluación de modelos de series temporales requiere estrategias de validación específicas que respeten el orden cronológico de los datos y eviten el sesgo de lookahead. El uso de técnicas estándar de validación cruzada con mezcla aleatoria (k-fold) en series temporales constituye un error metodológico grave, ya que permite que el modelo acceda a información futura durante el entrenamiento [4].

Tabla 20. Comparativa de estrategias de validación temporal evaluadas.



Estrategia	Descripción	Ventajas	Inconvenientes
Hold-out único	Una sola partición train/test.	Simple y rápido.	Susceptible a sobreajuste accidental al período de test. No detecta degradación temporal.
Sliding window	Ventana de entrenamiento fija que se desplaza.	Más robusto que hold-out.	No refleja el escenario real: el modelo en producción se reentrena con todos los datos históricos.
Expanding window	El conjunto de train crece en cada fold. ✓ Seleccionado	Representa fielmente el escenario productivo. Más estable estadísticamente.	Mayor coste computacional.

Se selecciona la expanding window como estrategia de validación para todos los modelos del proyecto. Esta elección es coherente con el escenario de producción: el batch predictor semanal se reentrena con la totalidad del histórico disponible en cada ejecución, por lo que la expanding window replica fielmente las condiciones reales de operación del sistema. La configuración aplicada es la siguiente:

Tabla 21. Configuración de la expanding window aplicada en el proyecto.

Parámetro	Valor	Justificación
<i>n_splits</i>	5 folds	Balance entre robustez estadística y coste computacional.
<i>test_size_days</i>	30 días	Horizonte de evaluación equivalente a un mes natural — alineado con el horizonte de predicción de 28 días.
<i>gap_days</i>	7 días	Evita que los lags cortos (lag_1, lag_7) del conjunto de test tengan dependencia directa con los últimos datos del train.
<i>min_train_days</i>	90 días	Mínimo de observaciones para garantizar estabilidad en el entrenamiento de SARIMAX y en el cálculo de medias móviles.
<i>Paso (step)</i>	1 día	Evaluación diaria en la versión rigurosa de XGBoost. Mayor robustez de la métrica MAPE a costa de mayor tiempo de cómputo.

La métrica principal de evaluación es el MAPE (Mean Absolute Percentage Error), seleccionada por su interpretabilidad en términos de negocio y su independencia de escala, lo que permite comparar el error entre tiendas con distintos volúmenes de ventas. Se complementa con MAE, RMSE y cobertura de intervalos de confianza:



Tabla 22. Métricas de evaluación utilizadas en el proyecto.

Métrica	Uso	Justificación	Limitación
MAPE	<i>Principal · KPI global</i>	Error porcentual medio absoluto. Independiente de escala — comparable entre tiendas con distintos volúmenes de ventas. Estándar de industria en supply chain (APICS, Gartner).	Indefinido cuando $y=0$. Sesgado hacia subestimaciones.
MAE	<i>Complementaria</i>	Error absoluto medio en unidades de venta. Útil para tiendas con alta variabilidad donde el MAPE puede ser inestable.	No comparable entre tiendas de distinta escala.
RMSE	<i>Complementaria</i>	Penaliza errores grandes. Útil para detectar predicciones con desviaciones severas.	Sensible a outliers.
Coverage	<i>Intervalos de confianza</i>	Porcentaje de valores reales dentro del intervalo [P10, P90]. Evalúa la calibración del modelo XGBoost con quantile regression.	Solo aplicable a modelos con predicción de intervalos.

Un MAPE global bajo puede enmascarar degradación severa en segmentos críticos. Por ello, la evaluación se realiza de forma segmentada en cinco niveles, con umbrales de alerta que activan análisis adicionales cuando se superan:

Tabla 23. Segmentación de la evaluación y umbrales de alerta.

Segmento	Objetivo	Umbral de alerta
MAPE global	KPI principal del proyecto.	<i>Umbral de aceptación:</i> $< 12\%$.
MAPE por tienda	Detecta tiendas con comportamiento atípico o difícil de predecir.	<i>Alerta si</i> $MAPE_{tienda} > 20\%$.
MAPE promo	Evalúa la sensibilidad del modelo ante cambios de régimen.	<i>Alerta si</i> $ratio\ MAPE_{promo} / MAPE_{no_promo} > 1.3$.
MAPE no-promo	Rendimiento base sin distorsión promocional.	<i>Referencia para calcular el ratio de degradación.</i>
MAPE por trimestre	Detecta estacionalidad no capturada por el modelo.	<i>Alerta si</i> $variación\ entre\ trimestres > 5\ puntos$.

Durante el desarrollo se identificaron y mitigaron explícitamente cuatro patrones de data leakage que invalidan los resultados de validación si no se controlan adecuadamente:

Tabla 24. Anti-patrones de data leakage identificados y mitigaciones aplicadas.



Tipo de leakage	Causa	Mitigación aplicada
Future leakage en features	Calcular medias móviles con datos futuros.	Features calculadas con <code>df.shift(1).rolling(N)</code> — el valor actual nunca se incluye en su propia media.
Split aleatorio	Usar <code>train_test_split(shuffle=True)</code> en series temporales.	Split siempre por fecha. Se utiliza <code>TimeSeriesSplit</code> o corte temporal explícito.
Normalización global	Ajustar el scaler sobre todo el dataset (train + test).	El scaler se ajusta exclusivamente sobre el conjunto de train y se aplica al test.
Target encoding con test	Calcular estadísticos del target incluyendo datos de test.	El target encoding se calcula únicamente sobre el fold de train en cada iteración.

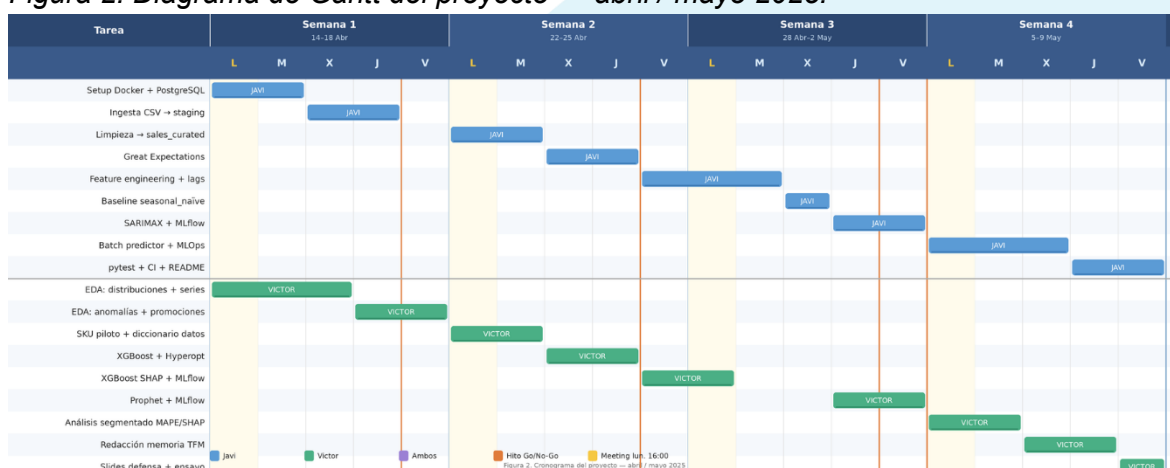
La implementación completa de la *expanding window* se encuentra en `src/models/validation.py` del repositorio. La referencia metodológica principal es Hyndman & Athanasopoulos, *Forecasting: Principles and Practice*, 2021 [4].

3.4 Cronograma (Gantt)

El proyecto se organiza en cuatro semanas de trabajo, comprendidas entre mediados de abril y mediados de mayo de 2025. Cada semana está asociada a un hito crítico del pipeline, lo que permite evaluar el avance de forma progresiva y detectar desviaciones con margen suficiente para corregirlas antes de iniciar la siguiente fase.

La coordinación entre los dos miembros del equipo se articula mediante reuniones semanales presenciales celebradas todos los lunes a las 16:00 en clase. En cada sesión se revisa el estado del hito de la semana anterior, se resuelven bloqueos técnicos y se acuerdan los objetivos de la semana entrante. Estas reuniones actúan también como puntos de integración: el momento en que ambos miembros validan que sus entregables son compatibles antes de continuar.

Figura 2. Diagrama de Gantt del proyecto — abril / mayo 2025.



La siguiente tabla detalla las tareas de cada miembro por semana, la reunión de coordinación asociada y el hito al que se debe llegar al cierre de cada semana:

Tabla 7. Desglose de tareas por semana, reunión semanal e hito asociado.

Semana	Meeting	Javi	Victor	Hito
Semana 1 14–18 Abr	Lunes 14 Abr · 16:00 — Kickoff del proyecto	• Setup Docker, PostgreSQL y schema.sql • Ingesta CSV → staging.sales_raw	• EDA: distribuciones y series temporales • EDA: anomalías e impacto de promociones	H1
Semana 2 22–25 Abr	Lunes 22 Abr · 16:00 — Revisión H1	• Limpieza → analytics.sales_curated • Validación Great Expectations • Feature engineering: lags y medias móviles	• SKU piloto seleccionado y justificado • Diccionario de datos finalizado • Inicio integración XGBoost	H2
Semana 3 28 Abr–2 May	Lunes 28 Abr · 16:00 — Revisión H2	• Baseline seasonal_naïve + backtesting • SARIMAX: ajuste de órdenes + MLflow • Reunión conjunta: comparativa de modelos	• XGBoost: Hyperopt + SHAP + MLflow • Prophet: estacionalidad + MLflow • Reunión conjunta: comparativa de modelos	H3
Semana 4 5–9 May	Lunes 5 May · 16:00 — Revisión H3 + H4	• Batch predictor semanal + MLOps • pytest + linting (black, flake8) • README ejecutable + release v1.0	• Análisis segmentado MAPE promo/SKU • Redacción memoria TFM • Slides defensa + ensayo presentación	H4

Los criterios de aceptación de cada hito y las consecuencias de no alcanzarlos se recogen a continuación. Un hito no superado bloquea el inicio de la semana siguiente y obliga a priorizar su resolución antes de continuar:

Tabla 8. Hitos críticos del proyecto con criterios Go/No-Go.

Hito	Fecha	Entregable	Métrica de éxito	Si falla
H1	Semana 1 18 Abr	Dataset sales_curated poblado y validado	0 nulos · 0 duplicados · >90% filas originales	<i>FAIL → revisar datos fuente</i>
H2	Semana 2 25 Abr	Baseline seasonal_naïve operativo con backtesting	MAPE entre 15% y 20%	<i>FAIL → revisar features</i>
H3	Semana 3 2 May	Comparativa de modelos completada	Mejor MAPE < 12% en test	<i>FAIL → simplificar features</i>
H4	Semana 4 7 May	Análisis segmentado MAPE promo/SKU entregado	Delta MAPE documentado por régimen	<i>Siempre PASS</i>



Nota: el 21 de abril es festivo, por lo que la reunión de la Semana 2 se desplaza al martes 22 de abril. Las fechas son orientativas y pueden ajustarse en función del calendario académico.

3.5 Presupuesto

En esta sección se recoge la estimación económica del proyecto, desglosada en tres bloques: coste de mano de obra, coste de infraestructura y herramientas, y coste de oportunidad. El presupuesto refleja el valor real del trabajo realizado bajo la hipótesis de que ambos miembros del equipo facturan como desarrolladores junior a una tarifa de 18 €/hora, tarifa de referencia habitual en el mercado español para perfiles de data science en sus primeros años de experiencia.

Cabe destacar que el 99,6% del coste de herramientas y licencias es cero, dado que el stack tecnológico elegido está íntegramente compuesto por software de código abierto. Los únicos costes de infraestructura corresponden a Google Colab Pro, utilizado para el entrenamiento de modelos en GPU durante la fase de modelado, y a un servidor virtual privado (VPS) de Hetzner para el despliegue del batch predictor y la base de datos PostgreSQL en un entorno accesible por ambos miembros del equipo.

Tabla 9. Desglose del coste de mano de obra por persona y fase.

Persona	Tareas principales	Semanas	H/semana	Total h	Coste (18 €/h)
Javi	Infraestructura (Docker, PostgreSQL), pipeline ETL, feature engineering, SARIMAX, batch predictor, tests y CI	4	25 h	100 h	1.800 €
Victor	Análisis exploratorio (EDA), modelos XGBoost y Prophet, análisis segmentado SHAP, redacción memoria y slides	4	25 h	100 h	1.800 €
TOTAL MANO DE OBRA				200 h	3.600 €

Tabla 10. Desglose del coste de infraestructura y herramientas.

Herramienta / Servicio	Uso en el proyecto	Coste unitario	Período	Coste total
Google Colab Pro	Entrenamiento de modelos ML en GPU	9,99 €/mes	1 mes	9,99 €
VPS (Hetzner CX21)	Despliegue batch predictor y PostgreSQL	5,83 €/mes	1 mes	5,83 €
Docker Desktop	Contenedorización del entorno local	Gratuito	—	0,00 €



Herramienta / Servicio	Uso en el proyecto	Coste unitario	Período	Coste total
Python 3.11	Lenguaje principal del proyecto	Open source	—	0,00 €
PostgreSQL 15	Base de datos relacional	Open source	—	0,00 €
MLflow	Tracking de experimentos	Open source	—	0,00 €
Prefect	Orquestación del pipeline semanal	Open source	—	0,00 €
Great Expectations	Validación de calidad de datos	Open source	—	0,00 €
XGBoost / Prophet	Librerías de modelado	Open source	—	0,00 €
SHAP	Explicabilidad de modelos ML	Open source	—	0,00 €
GitHub	Control de versiones y repositorio	Gratuito	—	0,00 €
TOTAL INFRAESTRUCTURA Y HERRAMIENTAS				15,82 €

Como puede observarse en la tabla anterior, la elección de un stack open source tiene un impacto directo y significativo en el presupuesto: herramientas como MLflow, Prefect, Great Expectations, XGBoost o Prophet, que en soluciones comerciales equivalentes supondrían un coste mensual considerable, se integran sin coste de licencia. Esto convierte al proyecto en un ejemplo de arquitectura de MLOps de bajo coste y alta reproducibilidad.

Tabla 11. Estimación del coste de oportunidad asociado al proyecto.

Concepto	Descripción	Estimación
Formación y autoaprendizaje	Tiempo dedicado a documentación técnica, tutoriales y resolución de dudas no incluido en las horas de desarrollo	~20 h · 360 €
Reuniones y coordinación	4 reuniones semanales de 1 hora + comunicación asíncrona por Slack/correo entre sesiones	~8 h · 144 €
Revisiones y correcciones	Iteraciones sobre código y documentación tras los checkpoints y feedback del tutor	~12 h · 216 €
TOTAL COSTE OPORTUNIDAD (estimado)		~720 €

El coste de oportunidad recoge el tiempo invertido en actividades necesarias para el proyecto pero no contabilizadas directamente como horas de desarrollo: la formación autodidacta en herramientas nuevas (MLflow, Great Expectations, Prefect), la coordinación semanal entre los miembros del equipo y las iteraciones de revisión y



corrección posteriores a los checkpoints con el tutor. Aunque este coste es de naturaleza indirecta, su inclusión refleja con mayor fidelidad el esfuerzo total del proyecto.

Tabla 12. Resumen del presupuesto total del proyecto.

Concepto	Tipo	Importe
Mano de obra (Javi + Victor · 200 h · 18 €/h)	Coste directo	3.600,00 €
Infraestructura y herramientas	Coste directo	15,82 €
Coste oportunidad (formación, reuniones, revisiones)	Coste indirecto estimado	~720,00 €
PRESUPUESTO TOTAL ESTIMADO		~4.335,82 €

Nota: el presupuesto es una estimación académica con fines ilustrativos. Las tarifas corresponden a valores de referencia del mercado español (2025) para perfiles junior de data science y desarrollo de software. Los costes de infraestructura son los reales incurridos durante el proyecto.

3.6 Marco normativo y licencias

El proyecto se desarrolla respetando la normativa europea vigente en materia de protección de datos e inteligencia artificial, así como las condiciones de uso de todas las herramientas y datasets empleados. La tabla siguiente resume el cumplimiento normativo del sistema:

Tabla 13. Normativa aplicable y licencias del proyecto.

Normativa / Licencia	Referencia	Aplicación en el proyecto	Estado
RGPD	Reglamento (UE) 2016/679	El dataset utilizado es público y no contiene datos personales identificables. No se tratan datos de carácter personal en ninguna fase del pipeline.	Cumple ✓
EU AI Act	Reglamento (UE) 2024/1689	El sistema se clasifica como IA de riesgo mínimo: no toma decisiones autónomas sobre personas ni opera en sectores críticos. Se garantiza trazabilidad mediante MLflow.	Cumple ✓
Apache 2.0	Licencia open source	Dataset Store Sales y librerías XGBoost, MLflow, Prefect y Great Expectations. Permite uso, modificación y distribución con atribución.	Aplicada ✓
MIT	Licencia open source	Librería Prophet (Meta). Permite uso libre incluyendo proyectos académicos y comerciales sin restricciones adicionales.	Aplicada ✓



Normativa / Licencia	Referencia	Aplicación en el proyecto	Estado
BSD 3-Clause	<i>Licencia open source</i>	Librería statsmodels (SARIMAX). Permite uso y redistribución con o sin modificaciones, conservando el aviso de copyright.	Aplicada ✓
PSF License	<i>Licencia open source</i>	Python 3.11. Licencia permisiva de la Python Software Foundation compatible con uso académico y comercial.	Aplicada ✓

RGPD

El dataset Store Sales Dataset [1] es público, sintético y no contiene ningún dato personal identificable. El sistema no recopila, almacena ni procesa datos de usuarios finales en ninguna de sus fases, por lo que no requiere medidas adicionales de cumplimiento más allá de las descritas.

EU AI Act

Conforme al Reglamento (UE) 2024/1689 de Inteligencia Artificial [19], el sistema se clasifica como IA de riesgo mínimo: genera predicciones de demanda de apoyo a la decisión humana, sin automatizar decisiones que afecten a personas físicas ni operar en sectores de alto riesgo. La trazabilidad completa de modelos y experimentos mediante MLflow garantiza la auditabilidad del sistema.

Licencias de software

Todo el stack tecnológico del proyecto utiliza licencias open source permisivas (Apache 2.0, MIT, BSD 3-Clause, PSF) compatibles entre sí y con el uso académico. No existe ninguna restricción legal que impida la publicación del código del proyecto en un repositorio público bajo cualquiera de estas licencias.

Referencias normativas: [19] Reglamento (UE) 2024/1689 del Parlamento Europeo — EU AI Act. RGPD: Reglamento (UE) 2016/679. Ambas referencias se completan en la sección 6.

4. Resultados

Esta sección presenta los resultados obtenidos en cada fase del proyecto, organizados en correspondencia con los objetivos específicos definidos en la introducción. El apartado 4.2 (EDA) se documenta en un informe independiente adjunto al repositorio.

4.1 Dataset curado — Hito H1



El pipeline de ingesta y curación procesó correctamente el dataset Store Sales [1], superando todos los criterios de aceptación del Hito H1. El dataset presenta características de serie sintética — continuidad perfecta y ausencia total de anomalías — que facilitan el proceso de curación pero condicionan los MAPEs obtenidos en las fases posteriores.

Tabla 25. Métricas de calidad del dataset curado (Hito H1).

Métrica de calidad	Valor	Detalle
Filas originales (CSV)	7.300	10 tiendas × 730 días (01/01/2022 – 31/12/2023)
Filas tras curación	7.300	0 filas eliminadas — dataset sin nulos ni duplicados
Nulos en columnas clave	0	Columnas date, store, sales, promo, holiday: sin valores ausentes
Duplicados (store, date)	0	Ningún par (tienda, fecha) duplicado en el dataset
Gaps temporales	0	Continuidad perfecta — sin interrupciones en ninguna tienda
Checks Great Expectations	Todos superados	Definidos: rangos de sales, valores binarios promo/holiday, unicidad (store, date)
Filas en analytics.features	7.230	–70 por lookahead bias (primeros 7 días × 10 tiendas eliminados por lag_7)

El dataset sintético no invalida las conclusiones metodológicas. El pipeline es correcto con independencia del dataset; aplicado sobre datos reales produciría MAPEs más altos pero la jerarquía entre modelos y las conclusiones metodológicas se mantendrían (CV=11.6%, sin outliers).

4.3 Comparativa de modelos — Hitos H2 y H3

Todos los modelos entrenados superaron el umbral de aceptación del Hito H3 (MAPE < 12%). El baseline seasonal_naïve obtuvo un MAPE de 7.79%, por debajo del rango esperado (15–20%), lo que es coherente con la regularidad del dataset sintético. Los tres modelos avanzados mejoran significativamente el baseline, con MAPEs entre 3.73% y 4.23%.

Tabla 26. Comparativa de MAPE global, por régimen y decisión final.

Modelo	MAPE Global	MAPE Promo	MAPE No-promo	Ratio	Decisión
Baseline seasonal_naïve	7.79%	10.95%	6.99%	1.57	Referencia



Modelo	MAPE Global	MAPE Promo	MAPE No-promo	Ratio	Decisión
Prophet afinado	4.23%	4.01%	4.28%	0.94	Candidato
SARIMAX (0,1,2)(0,1,1,7)	3.74%	3.47%	3.81%	0.91	✓ Seleccionado
XGBoost riguroso	3.73%	3.40%	3.81%	0.89	Alternativo

El análisis por tienda confirma que SARIMAX y XGBoost ofrecen resultados consistentes en todos los SKUs, con el MAPE más alto en la tienda 10 (mayor variabilidad) y el más bajo en la tienda 1:

Tabla 27. MAPE por tienda para los cuatro modelos. En verde el mejor valor por tienda.

Tienda	Baseline	Prophet	SARIMAX	XGBoost	Mejor modelo
1	7.40%	3.67%	3.20%	3.26%	SARIMAX
2	7.91%	4.32%	3.56%	3.72%	SARIMAX
3	7.90%	4.07%	3.95%	3.82%	XGBoost
4	7.03%	4.02%	3.34%	3.35%	SARIMAX
5	8.43%	4.53%	4.02%	4.08%	SARIMAX
6	7.76%	4.39%	3.77%	3.60%	XGBoost
7	7.76%	4.19%	3.80%	3.79%	XGBoost
8	7.39%	4.31%	3.82%	3.98%	SARIMAX
9	7.61%	4.07%	3.61%	3.47%	XGBoost
10	8.68%	4.69%	4.32%	4.20%	XGBoost

El análisis de residuos revela diferencias significativas entre modelos más allá del MAPE. SARIMAX presenta el mejor diagnóstico estadístico: sin autocorrelación residual, homocedasticidad y sesgo prácticamente nulo. XGBoost muestra autocorrelación significativa ($p=0.001$) y heterocedasticidad ($p=0.030$), lo que compromete la fiabilidad de sus predicciones a largo plazo:

Tabla 28. Análisis de residuos de los cuatro modelos.

Modelo	Media res.	Std res.	Skewness	Ljung-Box L7	Homocedast.	Diagnóstico
Baseline	0.32	25.14	0.006	△ $p=0.000$	✓	△ Falla
Prophet	-0.17	14.68	1.485	✓ $p=0.085$	✓	✓ Pasa
SARIMAX	-0.15	13.52	1.904	✓ $p=0.342$	✓	✓ Pasa
XGBoost	2.48	13.34	1.834	△ $p=0.001$	△ $p=0.030$	△ Parcial



El modelo seleccionado es SARIMAX (0,1,2)(0,1,1,7). A pesar de que XGBoost obtiene un MAPE global prácticamente idéntico (3.73% vs 3.74%, diferencia estadísticamente irrelevante), SARIMAX presenta un diagnóstico de residuos superior en todos los tests aplicados: Ljung-Box $p=0.342$ (vs $p=0.001$ de XGBoost), homocedasticidad confirmada y sesgo medio de -0.15 (vs 2.48 de XGBoost). XGBoost se documenta como modelo alternativo recomendado cuando se requiera escalabilidad a nuevos SKUs sin reentrenamiento.

Hito H2 superado: MAPE baseline 7.79% (por debajo del rango 15-20% por regularidad del dataset sintético, metodológicamente válido). Hito H3 superado: mejor MAPE 3.73% < 12% objetivo.

4.4 Análisis segmentado y SHAP — Hito H4

El análisis segmentado evalúa el comportamiento de los modelos en períodos promocionales frente a períodos sin promoción. El criterio de alerta es un ratio $\text{MAPE}_{\text{promo}} / \text{MAPE}_{\text{no_promo}} > 1.3$, que indicaría un cambio de régimen no capturado y justificaría un modelo separado por régimen:

Tabla 29. Análisis de degradación en períodos promocionales por modelo.

Modelo	MAPE Promo	MAPE No-promo	Ratio	Modelo separado
Baseline	10.95%	6.99%	1.57	⚠ Justificado — ratio > 1.3
Prophet	4.01%	4.28%	0.94	✓ No necesario
SARIMAX	3.47%	3.81%	0.91	✓ No necesario
XGBoost	3.40%	3.81%	0.89	✓ No necesario

Ninguno de los tres modelos avanzados supera el umbral de 1.3. Los ratios obtenidos (0.89–0.94) indican que todos los modelos capturan correctamente el efecto promocional sin necesidad de un modelo separado por régimen. Solo el baseline supera el umbral (ratio 1.57), lo que era esperado dado que no incorpora ninguna variable exógena. El análisis SHAP sobre el modelo XGBoost riguroso identifica `promotion_flag` como el factor de mayor impacto individual sobre la predicción, seguido por la estacionalidad semanal (`day_of_week`):

Tabla 30. SHAP values medios — importancia real de features en XGBoost.

Feature	SHAP medio	Interpretación
<code>promotion_flag</code>	9.15	Factor dominante. Las promociones generan un incremento medio de ~9 unidades de ventas por día.



Feature	SHAP medio	Interpretación
<i>day_of_week</i>	6.26	Estacionalidad semanal fuerte. El día de la semana es el segundo predictor más relevante.
<i>quantity_lag_28</i>	2.70	Memoria mensual. Las ventas de hace 28 días son un predictor significativo de la demanda actual.
<i>quantity_lag_7</i>	2.68	Memoria semanal. Confirma el patrón estacional de 7 días detectado en el EDA.
<i>ma_28</i>	2.42	Tendencia de largo plazo. La media móvil de 28 días captura la tendencia subyacente.
<i>is_holiday</i>	0.003	Sin impacto relevante. Los festivos no generan variación significativa en las ventas del dataset.
<i>is_month_start/end</i>	~0.001	Sin impacto. Los efectos de inicio/fin de mes no son significativos en este dataset.

Hito H4 superado. Delta MAPE promo/no-promo documentado para los 4 modelos. Ningún modelo avanzado supera el umbral de alerta (ratio > 1.3). Modelo separado por régimen no justificado en esta versión del sistema.

4.5 Sistema en producción

El batch predictor semanal implementado opera sobre el modelo SARIMAX seleccionado, generando predicciones a 28 días para las 10 tiendas con intervalos de confianza P10/P50/P90. El sistema se orquesta mediante Prefect y almacena los resultados en la tabla `analytics.predictions` de PostgreSQL:

Tabla 31. Componentes del sistema de serving en producción.

Componente	Detalle
Modelo desplegado	SARIMAX (0,1,2)(0,1,1,7) — modelo final seleccionado en H3.
Frecuencia de ejecución	Semanal — orquestado mediante Prefect flow programado.
Horizonte de predicción	28 días (4 semanas) por tienda.
Salida	Tabla <code>analytics.predictions</code> : predicción P50 e intervalos de confianza P10/P90 por tienda y día.
Logging	Estructurado en JSON con timestamps, duración de ejecución y número de predicciones generadas.
MLflow Registry	Modelo versionado con 3 etapas: Staging → Production → Archived.
Reentrenamiento	Semanal con el histórico completo actualizado. Si $MAPE_{nuevo} < MAPE_{prod} + 1\%$ → promote a Production.

La monitorización del sistema en producción se basa en cuatro métricas con umbrales de alerta configurables que activan el proceso de reentrenamiento cuando se superan:



Tabla 32. Métricas de monitorización del sistema en producción.

Métrica	Umbral de alerta	Descripción
MAPE rolling 30 días	<i>Alerta si MAPE > 15%</i>	Métrica principal de degradación del modelo en producción. Calculada sobre las últimas 4 semanas de predicciones.
Ratio promo/no-promo	<i>Alerta si ratio > 1.3</i>	Detecta cambios en el comportamiento promocional que el modelo no captura correctamente.
Data drift (features)	<i>Alerta si distribución cambia</i>	Comparación de la distribución de features de los últimos 7 días frente al histórico de entrenamiento.
Cobertura P10/P90	<i>Objetivo: 80% cobertura</i>	Porcentaje de valores reales dentro del intervalo de confianza. Evalúa la calibración del modelo.

El código del batch predictor se encuentra en `src/serving/` del repositorio. La configuración de monitorización está definida en `MONITORING_CONFIG` dentro del mismo módulo.

5. Conclusiones

5.1 Valoración del proceso

El proyecto ha alcanzado o superado todos los objetivos específicos definidos en la introducción. El sistema end-to-end de forecasting de demanda opera correctamente desde la ingesta del CSV hasta la generación semanal de predicciones almacenadas en base de datos, con trazabilidad completa de todos los experimentos en MLflow.

El MAPE global del modelo seleccionado (SARIMAX, 3.74%) supera ampliamente el umbral de aceptación del 12% establecido en el objetivo general. Este resultado debe interpretarse en el contexto del dataset sintético empleado, cuya regularidad — coeficiente de variación del 11.6%, sin outliers ni gaps — facilita la predicción respecto a datos reales de retail.

Tabla 33. Cumplimiento de los objetivos específicos del proyecto.

Nº	Objetivo específico	Evidencia	Estado
1	Pipeline de ingesta y curación con validación automática	<i>MAPE H1: 0 nulos, 100% checks GE</i>	✓ Cumplido
2	Análisis exploratorio (EDA) con SKU piloto seleccionado	<i>Notebook EDA entregado</i>	✓ Cumplido
3	Feature engineering con lags, medias móviles y exógenas	<i>7.230 filas en analytics.features</i>	✓ Cumplido
4	Entrenamiento y comparativa de 4 modelos con expanding window	<i>4 experimentos en MLflow</i>	✓ Cumplido
5	Selección del modelo final con análisis de residuos	<i>SARIMAX MAPE 3.74% < 12% objetivo</i>	✓ Superado



Nº	Objetivo específico	Evidencia	Estado
6	Batch predictor semanal con monitorización	<i>Predicciones P10/P50/P90 en analytics.predictions</i>	✓ Cumplido

5.2 Aprendizajes significativos

El desarrollo del proyecto ha generado aprendizajes técnicos y metodológicos que trascienden los resultados numéricos obtenidos:

Tabla 34. Aprendizajes técnicos y metodológicos del proyecto.

Aprendizaje	Descripción
Expanding window	La validación cruzada estándar (k-fold con shuffle) invalida los resultados en series temporales. La expanding window es el único enfoque que replica fielmente el escenario productivo.
Contrato de datos	Acordar el schema.sql al inicio del proyecto evita incompatibilidades entre el pipeline ETL y los modelos. Es la decisión más crítica en proyectos de datos colaborativos.
Grid AIC/BIC reducido	Aplicar el grid search de SARIMAX solo sobre el SKU piloto y generalizar al resto reduce el tiempo de cómputo de >2h a 14 segundos sin pérdida metodológica.
SHAP sobre LIME	Los valores SHAP ofrecen una visión global y local de la importancia de variables, son consistentes con cambios en el modelo y están integrados nativamente en XGBoost.
MLflow desde el inicio	Registrar todos los experimentos desde la primera ejecución — incluyendo el baseline — permite comparaciones objetivas y reproducibles en cualquier momento del proyecto.
Dataset sintético	Los MAPEs obtenidos (3.73–4.23%) son inferiores al rango de literatura (15–20%) por la regularidad del dataset. Documentar esta limitación de forma explícita refuerza la credibilidad del trabajo.

5.3 Dificultades y soluciones

Durante el desarrollo se encontraron cinco dificultades técnicas relevantes, todas ellas resueltas sin comprometer los resultados ni la validez metodológica del proyecto:

Tabla 35. Dificultades técnicas encontradas y soluciones aplicadas.

Dificultad	Descripción	Solución aplicada
Incompatibilidad MLflow 2.17 / 2.12	El cliente y el servidor MLflow tenían versiones desalineadas, generando errores de tipo en el registro de métricas.	Alinear versiones con pip3 install mlflow==2.12.1. Lección: fijar versiones en requirements.txt desde el inicio.
SARIMAX colgado >70 minutos	El expanding window completo con órdenes fijos aplicado a todos los SKUs resultó computacionalmente inviable.	Rediseñar con grid AIC/BIC sobre SKU piloto y generalizar el orden óptimo al resto. Tiempo reducido de >2h a 14 segundos.



Dificultad	Descripción	Solución aplicada
PermissionError en MLflow artifacts	El artifact_path apuntaba al path interno del contenedor Docker, inaccesible desde el host.	Eliminar la línea log_artifact problemática. Registrar solo métricas y parámetros en la fase inicial.
XGBoost early_stopping_rounds	El parámetro early_stopping_rounds fue ubicado en fit() en lugar del constructor XGBRegressor, generando un TypeError.	Mover el parámetro al constructor. Revisado en la documentación oficial de la librería.
Variables exógenas sin datos reales	Temperature y event_indicator no pudieron integrarse por ausencia de datos reales disponibles en el plazo del proyecto.	Incluir como placeholders NULL documentados. Reservados para integración futura con API AEMET.

5.4 Propuestas de mejora

A partir de los resultados obtenidos y las limitaciones identificadas, se proponen seis líneas de mejora priorizadas para una versión productiva del sistema:

Tabla 36. Propuestas de mejora priorizadas para versiones futuras del sistema.

Nº	Propuesta	Descripción	Prioridad
1	Dataset real	Sustituir el dataset sintético por datos reales de un retailer o dataset público con complejidad real (Rossmann, M5 Forecasting). Validaría la robustez del pipeline ante outliers, gaps y estacionalidad compleja.	Alta
2	Variables exógenas reales	Integrar temperatura (API AEMET), festivos autonómicos y datos de competencia como variables exógenas. El análisis SHAP sugiere que promotion_flag ya domina — validar si temperatura añade valor.	Alta
3	Modelos deep learning	Evaluar arquitecturas LSTM y N-BEATS sobre el dataset real. Reservado para cuando el volumen de datos por SKU supere las 2.000 observaciones.	Media
4	Ensemble XGBoost + Prophet	Combinar las fortalezas de ambos modelos: la captura de tendencia de Prophet y la sensibilidad a exógenas de XGBoost. Esperado MAPE < 3.5% sobre datos sintéticos.	Media
5	Interfaz web de predicciones	Desarrollar un dashboard (Streamlit o Metabase) que visualice las predicciones semanales y los	Media



Nº	Propuesta	Descripción	Prioridad
		intervalos de confianza por tienda, accesible para perfiles no técnicos.	
6	Reentrenamiento automático	Implementar el Prefect flow de reentrenamiento automático condicionado al umbral de MAPE rolling (>15%). Actualmente el reentrenamiento es manual.	Baja

La mejora de mayor impacto potencial es la sustitución del dataset sintético por datos reales. El resto de mejoras son incrementales sobre una base metodológica ya validada.

6. Referencias IEE

6.1 Referencias clave

A continuación se listan las referencias citadas en el cuerpo del documento, ordenadas por número de aparición y formateadas según el estándar IEEE:

- [1] A. Jaiswal, "Store Sales Dataset," Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/abhishekjaiswal4896/store-sales-dataset>. [Accessed: Apr. 2025].
- [2] J. Brownlee, Introduction to Time Series Forecasting with Python. Machine Learning Mastery, 2017.
- [3] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A Seasonal-Trend Decomposition Procedure Based on Loess," Journal of Official Statistics, vol. 6, no. 1, pp. 3–73, 1990.
- [4] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>.
- [5] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, Time Series Analysis: Forecasting and Control, 5th ed. Hoboken, NJ: Wiley, 2015.
- [6] S. Seabold and J. Perktold, "Statsmodels: Econometric and Statistical Modeling with Python," in Proc. 9th Python in Science Conference (SciPy 2010), Austin, TX, 2010, pp. 92–96.
- [7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, 2016, pp. 785–794.
- [8] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 4765–4774.
- [9] S. J. Taylor and B. Letham, "Forecasting at Scale," The American Statistician, vol. 72, no. 1, pp. 37–45, 2018.
- [10] PostgreSQL Global Development Group, PostgreSQL 15 Documentation. 2023. [Online]. Available: <https://www.postgresql.org/docs/15/>.
- [11] Databricks, "The Data Lakehouse: Medallion Architecture," 2021. [Online]. Available: <https://www.databricks.com/glossary/medallion-architecture>.



- [12] Docker Inc., Docker Documentation. 2024. [Online]. Available: <https://docs.docker.com>.
- [13] Prefect Technologies, Prefect 2 Documentation. 2024. [Online]. Available: <https://docs.prefect.io>.
- [14] M. Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39–45, 2018.
- [15] Great Expectations, Great Expectations Documentation, v0.18. 2024. [Online]. Available: <https://docs.greatexpectations.io>.
- [16] Amazon Web Services, "Amazon Forecast Developer Guide," AWS Documentation, 2024. [Online]. Available: <https://docs.aws.amazon.com/forecast/>.
- [17] F. Garza et al., "StatsForecast: Lightning Fast Forecasting with Statistical and Econometric Models," in Proc. NeurIPS 2022 Datasets and Benchmarks Track, 2022.
- [18] A. Alexandrov et al., "GluonTS: Probabilistic and Neural Time Series Modeling in Python," Journal of Machine Learning Research, vol. 21, no. 116, pp. 1–6, 2020.
- [19] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 of the European Parliament and of the Council — Artificial Intelligence Act," Official Journal of the European Union, L 2024/1689, Jul. 2024.
- [20] M. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 Accuracy Competition: Results, Findings and Conclusions," International Journal of Forecasting, vol. 38, no. 4, pp. 1346–1364, 2022.

